

1 Structure du dossier de l'IA

Le projet de l'IA est organisé de la manière suivante :

IA/

```
    frontend/
        app.js
        index.html
        style.css

    backend/
        data/
            fichiers PDF      indexer

    node_modules/

    utils/
        chunking.js
        retrieval.js
        similarity.js

    .env
    context.js
    embeddings.json
    index-document.js
    package.json
    package-lock.json
    server.js
```

2 Frontend

2.1 app.js

Le fichier `app.js` contient le code JavaScript permettant de communiquer avec le backend de l'IA.

Lors de l'intégration au frontend du site, ce fichier devra être adapté à la structure du site.

Il faudra notamment modifier :

- la fonction située à la ligne 9;
- la ligne 23;
- la ligne 39.

Ces modifications permettent d'adapter les requêtes et les éléments du frontend à ceux du site final.

2.2 index.html

Le fichier `index.html` sert uniquement à tester l'interface de l'IA.

2.3 style.css

Le fichier `style.css` contient les styles utilisés par l'interface de test.

3 Backend

Le dossier `backend` contient le serveur Node.js ainsi que les différents fichiers nécessaires au fonctionnement du système RAG et à l'indexation des documents.

3.1 data

Le dossier `data` contient les fichiers PDF qui seront indexés par l'IA.

Si nécessaire, ce dossier pourra être remplacé par le dossier `resources` directement présent dans le backend du site.

3.2 node_modules

Le dossier `node_modules` contient les dépendances installées par npm.

Ce dossier est généré automatiquement lors de l'exécution de :

```
npm install
```

Il n'est donc pas nécessaire de le créer ou de le télécharger manuellement.

3.3 utils

Le dossier `utils` contient les différentes fonctions utilisées par le système de recherche de l'IA.

3.3.1 chunking.js

Le fichier `chunking.js` contient la fonction permettant de découper le texte des documents en différents chunks.

3.3.2 retrieval.js

Le fichier `retrieval.js` contient la fonction permettant de récupérer les chunks les plus pertinents en fonction de la question posée par l'utilisateur.

3.3.3 similarity.js

Le fichier `similarity.js` contient la fonction permettant de calculer la similarité entre deux vecteurs.

3.4 context.js

Le fichier `context.js` contient le system prompt de l'IA.

Il permet de définir les directives, le comportement et les informations générales que l'IA doit connaître.

On peut notamment y ajouter les informations qui ne sont pas suffisamment longues pour nécessiter un document PDF.

3.5 `embeddings.json`

Le fichier `embeddings.json` est généré automatiquement par le script `index-document.js`.

Il contient les chunks issus des documents ainsi que leurs vecteurs d'embedding.

Il n'est donc pas nécessaire de modifier ce fichier manuellement.

3.6 `index-document.js`

Le fichier `index-document.js` contient le script permettant d'indexer les fichiers PDF.

Le script :

1. lit les fichiers PDF ;
2. extrait leur texte ;
3. découpe le texte en chunks ;
4. génère les embeddings ;
5. sauvegarde les résultats dans `embeddings.json`.

3.7 `package.json`

Le fichier `package.json` contient les dépendances du projet ainsi que les scripts permettant notamment de lancer le serveur et d'effectuer l'indexation des documents.

3.8 `package-lock.json`

Le fichier `package-lock.json` est généré automatiquement par npm.

Il contient les versions exactes des dépendances utilisées par le projet.

3.9 `server.js`

Le fichier `server.js` contient le serveur Express.

Il reçoit les requêtes provenant du frontend, traite les questions de l'utilisateur à l'aide du système RAG et envoie ensuite la réponse de l'IA au frontend.

4 Installation

Le projet utilise Node.js.

Si Node.js n'est pas installé sur votre machine, il faut l'installer avant de continuer.

Pour vérifier que Node.js et npm sont correctement installés, exécutez :

```
node —version  
npm —version
```

5 Installation des dépendances

Après avoir cloné le repository depuis GitHub, ouvrez un terminal dans le dossier du projet.

Accédez ensuite au dossier `backend` :

```
cd backend
```

Installez les dépendances avec :

```
npm install
```

Cette commande installe automatiquement toutes les dépendances déclarées dans le fichier `package.json` et génère le dossier `node_modules`.

5.1 pdf-parse

Si le module `pdf-parse` n'est pas déjà présent dans les dépendances du projet, installez-le avec :

```
npm install pdf-parse
```

Si `pdf-parse` est déjà déclaré dans le `package.json`, cette commande supplémentaire n'est pas nécessaire.

6 Indexation des documents

Avant de lancer le serveur, les documents PDF doivent être indexés.

Placez les fichiers PDF à indexer dans :

```
backend/data/
```

Depuis le dossier `backend`, exécutez :

```
node index-document.js
```

Cette commande va traiter les documents et générer le fichier :

```
embeddings.json
```

Si de nouveaux PDF sont ajoutés ou si les documents existants sont modifiés, il faut relancer le script d'indexation.

7 Lancer le serveur

Pour lancer le serveur, assurez-vous d'être dans le dossier `backend`.

Exécutez :

```
node server.js
```

Le serveur Express démarre alors et peut recevoir les requêtes envoyées par le frontend.